

14 Spark Optimization

Spark Optimization - Comprehensive Performance Guide

Overview

Spark optimization is a **multi-faceted discipline** that encompasses data layout, resource management, query planning, and execution strategies. Effective optimization can yield **10x-100x performance improvements** in real-world scenarios.

Core Principle: Optimization in Spark is about minimizing data movement, maximizing parallelism, and leveraging the engine's built-in capabilities.

Optimization Framework

Performance Hierarchy

1. Data Layout & Format (Highest Impact)
2. Resource Allocation & Configuration
3. Query & Execution Optimization
4. Code-Level Optimizations (Lowest Impact)

Optimization Categories

1. **Data-Centric Optimizations** - Layout, format, and partitioning
2. **Execution Engine Optimizations** - Catalyst, Tungsten, and AQE
3. **Resource Management** - Memory, CPU, and cluster configuration
4. **Code-Level Optimizations** - API usage and algorithm choices

Data-Centric Optimizations

1. Partitioning Strategies

Concept and Impact

Partitioning is the foundation of Spark's parallel processing model. Proper partitioning ensures:

- Balanced workload distribution
- Minimized data shuffling
- Optimal task parallelism
- Reduced memory pressure

Types of Partitioning

Hash Partitioning

```
# Distribute data based on hash of key
df_hash_partitioned = df.repartition(10, "user_id")
```

Range Partitioning

```
# Distribute data based on range of values
df_range_partitioned = df.repartitionByRange(10, "timestamp")
```

Custom Partitioning

```
# Custom partitioner for specific use cases
from pyspark.sql.functions import col
df_custom = df.repartition(col("year"), col("month"))
```

Partitioning Best Practices

Scenario	Recommended Approach	Rationale
Large datasets	2-3x number of cores	Ensures parallelism without overhead
Skewed data	Custom partitioning + salting	Prevents hot spots
Time-series data	Range partitioning by time	Supports efficient filtering
Join operations	Partition both tables on join key	Eliminates shuffle

Advanced Partitioning Techniques

Salting for Skewed Data

```
from pyspark.sql.functions import rand, concat, lit

# Add salt to skewed key
df_saltd = df.withColumn("salted_key",
    concat(col("skewed_key"), lit("_"), (rand() * 100).cast("int")))
df_partitioned = df_saltd.repartition("salted_key")
```

Bucketing for Repeated Joins

```
# Create bucketed tables for efficient joins
df.write \
    .bucketBy(10, "join_key") \
    .sortBy("sort_key") \
    .option("path", "bucketed_table") \
    .saveAsTable("bucketed_table")
```

2. Storage Format Optimization

Columnar Formats Comparison

Format	Compression	Query Speed	Write Speed	Schema Evolution
Parquet	Excellent	Very Fast	Moderate	Good
ORC	Excellent	Very Fast	Moderate	Good
Delta	Good	Fast	Fast	Excellent
Avro	Good	Moderate	Fast	Excellent

Parquet Optimization

```
# Optimized Parquet writing
df.write \
    .option("compression", "snappy") \
    .option("parquet.block.size", "268435456") \
    .option("parquet.page.size", "1048576") \
    .mode("overwrite") \
    .parquet("optimized_path")
```

Advanced Format Features

Predicate Pushdown

```
# Automatic predicate pushdown with Parquet
df_filtered = spark.read.parquet("data.parquet") \
    .filter(col("date") >= "2023-01-01") \
    .filter(col("category") == "electronics")
```

Partition Pruning

```
# Partitioned data structure
# /data/year=2023/month=01/day=01/file.parquet
df_pruned = spark.read.parquet("data") \
    .filter(col("year") == 2023) \
    .filter(col("month") == 1)
```

3. Caching and Persistence Strategies

Storage Level Selection

Storage Level	Memory Usage	CPU Usage	Fault Tolerance	Use Case
MEMORY_ONLY	High	Low	None	Fast, small datasets
MEMORY_AND_DISK	Medium	Low	Good	Balanced performance
MEMORY_ONLY_SER	Low	High	None	Memory-constrained
DISK_ONLY	None	Medium	Excellent	Large, infrequent access

Intelligent Caching Strategies

```
from pyspark import StorageLevel

# Cache frequently accessed data
df_frequent = df.filter(col("status") == "active")
df_frequent.persist(StorageLevel.MEMORY_AND_DISK_SER)

# Cache intermediate results in complex pipelines
df_transformed = df.select(...).filter(...).groupBy(...)
df_transformed.cache()

# Use in multiple operations
result1 = df_transformed.agg(...)
result2 = df_transformed.join(...)
```

Cache Management

```
# Monitor cache usage
spark.catalog.isCached("table_name")

# Clear cache when no longer needed
df.unpersist()
spark.catalog.clearCache()
```

Execution Engine Optimizations

1. Catalyst Optimizer

Query Plan Optimization

The Catalyst optimizer applies **rule-based** and **cost-based** optimizations:

```
# View execution plan
df.explain(True) # Shows all optimization phases
df.explain("cost") # Shows cost-based optimization
```

Optimization Rules Examples

- **Predicate Pushdown:** Filters applied at data source
- **Projection Pushdown:** Only required columns read
- **Constant Folding:** Compile-time expression evaluation
- **Join Reordering:** Optimal join sequence

2. Tungsten Execution Engine

Memory Management

```
# Tungsten optimizations are automatic with DataFrames
df_optimized = df.select(
    col("id"),
    col("name"),
    (col("price") * col("quantity")).alias("total")
)
```

Code Generation

```
# Whole-stage code generation example
df.groupBy("category") \
    .agg(
        sum("sales").alias("total_sales"),
        avg("price").alias("avg_price"),
        count("*").alias("count")
    ) \
    .show()
```

3. Adaptive Query Execution (AQE)

Configuration and Benefits

```
# Enable AQE
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
```

AQE Optimizations

Dynamic Partition Coalescing

```
# AQE automatically reduces partition count
spark.conf.set("spark.sql.adaptive.coalescePartitions.minPartitionNum", "1")
spark.conf.set("spark.sql.adaptive.coalescePartitions.initialPartitionNum", "200")
```

Skew Join Optimization

```
# AQE detects and handles skewed joins
spark.conf.set("spark.sql.adaptive.skewJoin.skewedPartitionFactor", "5")
spark.conf.set("spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes", "256MB")
```

4. Join Optimization Strategies

Join Type Selection

Broadcast Join

```
from pyspark.sql.functions import broadcast

# Explicit broadcast for small tables
df_result = large_df.join(
    broadcast(small_df),
    large_df.id == small_df.id
)

# Configure broadcast threshold
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "10MB")
```

Bucket Join

```
# Pre-bucketed tables for efficient joins
df1.write.bucketBy(10, "join_key").saveAsTable("table1")
df2.write.bucketBy(10, "join_key").saveAsTable("table2")

# Join bucketed tables (no shuffle)
result = spark.table("table1").join(spark.table("table2"), "join_key")
```

Sort-Merge Join

```
# Optimize for large-large table joins
df1_sorted = df1.repartition("join_key").sortWithinPartitions("join_key")
df2_sorted = df2.repartition("join_key").sortWithinPartitions("join_key")
result = df1_sorted.join(df2_sorted, "join_key")
```

Join Optimization Patterns

Star Schema Optimization

```
# Dimension table broadcasting
fact_table.join(broadcast(dim_date), "date_id") \
    .join(broadcast(dim_product), "product_id") \
    .join(broadcast(dim_customer), "customer_id")
```

Denormalization Strategy

```
# Pre-join frequently used tables
customer_orders = customers.join(orders, "customer_id")
customer_orders.write.mode("overwrite").saveAsTable("customer_orders_denorm")
```

Resource Management

1. Memory Configuration

Executor Memory Allocation

```
# Optimal memory configuration
spark.conf.set("spark.executor.memory", "8g")
spark.conf.set("spark.executor.memoryOverhead", "1g")
spark.conf.set("spark.memory.fraction", "0.8")
spark.conf.set("spark.memory.storageFraction", "0.3")
```

Memory Monitoring

```
# Monitor memory usage
spark.sparkContext.statusTracker().getExecutorInfos()
```

2. CPU and Parallelism

Core Allocation Strategy

```
# Optimal core allocation
spark-submit \
  --executor-cores 4 \
  --executor-memory 8g \
  --num-executors 10 \
  your_application.py
```

Parallelism Tuning

```
# Set optimal parallelism
spark.conf.set("spark.default.parallelism", "400")
spark.conf.set("spark.sql.shuffle.partitions", "200")
```

3. Cluster Configuration

Dynamic Allocation

```
# Enable dynamic allocation
spark.conf.set("spark.dynamicAllocation.enabled", "true")
spark.conf.set("spark.dynamicAllocation.minExecutors", "1")
spark.conf.set("spark.dynamicAllocation.maxExecutors", "20")
spark.conf.set("spark.dynamicAllocation.initialExecutors", "5")
```

Resource Isolation

```
# Configure resource isolation
spark.conf.set("spark.task.cpus", "1")
spark.conf.set("spark.executor.cores", "4")
spark.conf.set("spark.executor.instances", "10")
```

Code-Level Optimizations

1. API Selection and Usage

DataFrame vs RDD Performance

```
# Preferred: DataFrame API (optimized)
df.groupBy("category").agg(sum("sales"))

# Avoid: RDD API (unoptimized)
rdd.map(lambda x: (x.category, x.sales)).reduceByKey(lambda a, b: a + b)
```

Built-in Functions vs UDFs

```
from pyspark.sql.functions import when, col, regexp_replace

# Preferred: Built-in functions
df.withColumn("clean_text", regexp_replace(col("text"), "[^a-zA-Z0-9]", ""))

# Avoid: Python UDFs
@udf("string")
def clean_text_udf(text):
    return re.sub("[^a-zA-Z0-9]", "", text)
```

2. Advanced UDF Optimization

Vectorized UDFs (Pandas UDFs)

```
from pyspark.sql.functions import pandas_udf
from pyspark.sql.types import DoubleType
import pandas as pd

@pandas_udf(returnType=DoubleType())
def vectorized_calculation(prices: pd.Series, quantities: pd.Series) -> pd.Series:
    return prices * quantities * 0.9 # Apply discount

df.withColumn("discounted_total", vectorized_calculation(col("price"), col("quantity")))
```

Broadcast Variables and Accumulators

```
# Broadcast lookup tables
lookup_dict = {"A": 1, "B": 2, "C": 3}
broadcast_lookup = spark.sparkContext.broadcast(lookup_dict)

@udf("int")
def lookup_udf(key):
    return broadcast_lookup.value.get(key, 0)

# Use accumulators for metrics
error_count = spark.sparkContext.accumulator(0)
```

3. Shuffle Optimization

Reduce Shuffle Operations

```
# Minimize shuffles with proper partitioning
df_partitioned = df.repartition("key")

# Use reduceByKey instead of groupByKey
rdd.map(lambda x: (x.key, x.value)).reduceByKey(lambda a, b: a + b)
```

Map-Side Combine

```
# Combine before shuffle
df.groupBy("category").agg(
    sum("sales").alias("total_sales"),
    count("*").alias("count")
)
```

4. Serialization Optimization

Kryo Serialization

```
# Configure Kryo serialization
spark.conf.set("spark.serializer", "org.apache.spark.serializer.KryoSerializer")
spark.conf.set("spark.kryo.registrationRequired", "false")
```

Avoid Large Objects in UDFs

```
# Bad: Large objects in UDFs
large_object = create_large_object()
@udf("string")
def process_with_large_object(data):
    return large_object.process(data)

# Good: Broadcast large objects
broadcast_object = spark.sparkContext.broadcast(large_object)
@udf("string")
def process_with_broadcast(data):
    return broadcast_object.value.process(data)
```

Advanced Optimization Techniques

1. Skew Handling Strategies

Salting Technique

```
from pyspark.sql.functions import rand, concat, lit

# Add salt to skewed keys
df_salted = df.withColumn("salted_key",
    concat(col("skewed_key"), lit("_"), (rand() * 100).cast("int")))

# Process with salted keys
result = df_salted.groupBy("salted_key").agg(sum("value"))

# Remove salt from final result
final_result = result.withColumn("original_key",
    regexp_replace(col("salted_key"), "_\\d+$", ""))
```

Isolated Broadcast Join

```
# Separate skewed and non-skewed data
skewed_keys = ["popular_key1", "popular_key2"]
df_skewed = df.filter(col("key").isin(skewed_keys))
df_normal = df.filter(~col("key").isin(skewed_keys))

# Handle differently
result_normal = df_normal.join(other_df, "key")
```



```
result_skewed = df_skewed.join(broadcast(other_df), "key")

# Combine results
final_result = result_normal.union(result_skewed)
```

2. Dynamic Partition Pruning Configuration

```
# Enable dynamic partition pruning
spark.conf.set("spark.sql.optimizer.dynamicPartitionPruning.enabled", "true")
spark.conf.set("spark.sql.optimizer.dynamicPartitionPruning.reuseBroadcastOnly", "true")
```

Star Schema Optimization

```
# Fact table with partitioned dimensions
fact_table = spark.read.parquet("fact_sales") # Partitioned by date
dim_table = spark.read.parquet("dim_products") # Small dimension

# Dynamic pruning eliminates unnecessary partitions
result = fact_table.join(
    dim_table.filter(col("category") == "electronics"),
    "product_id"
)
```

3. Speculative Execution Configuration

```
# Enable speculative execution
spark.conf.set("spark.speculation", "true")
spark.conf.set("spark.speculation.interval", "100ms")
spark.conf.set("spark.speculation.multiplier", "1.5")
spark.conf.set("spark.speculation.quantile", "0.75")
```

4. Data Locality Optimization Preferred Locations

```
# Use locality-aware operations
df.rdd.preferredLocations # Check data locality
```

Rack-aware Scheduling

```
# Configure rack awareness
spark.conf.set("spark.sql.adaptive.localShuffleReader.enabled", "true")
```

Performance Monitoring and Tuning

1. Spark UI Analysis

Key Metrics to Monitor

- **Task Duration:** Identify slow tasks
- **Shuffle Read/Write:** Minimize data movement
- **GC Time:** Optimize memory usage

- **Data Skew:** Balance partition sizes

Stage-Level Analysis

```
# Access stage information
stage_info = spark.sparkContext.statusTracker().getStageInfo(stage_id)
```

2. Profiling Tools

Built-in Profilers

```
# Enable SQL execution profiling
spark.conf.set("spark.sql.execution.profiling.enabled", "true")
```

Custom Metrics

```
# Track custom metrics
from pyspark.util import AccumulatorParam

class ListAccumulatorParam(AccumulatorParam):
    def zero(self, value):
        return []

    def addInPlace(self, list1, list2):
        list1.extend(list2)
        return list1

metrics_accumulator = spark.sparkContext.accumulator([], ListAccumulatorParam())
```

3. Automated Tuning

Adaptive Configurations

```
# Let Spark adapt configurations
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
```

Best Practices and Anti-Patterns



Best Practices

1. Use **DataFrames/Datasets** over RDDs
2. Leverage **built-in functions** over UDFs
3. **Cache strategically** based on usage patterns
4. **Partition data** according to query patterns
5. **Monitor and measure** performance regularly
6. Use **columnar formats** for analytical workloads
7. **Enable AQE** for dynamic optimization
8. **Broadcast small tables** in joins
9. **Avoid data skew** through proper partitioning
10. **Size executors appropriately** for workload



Common Anti-Patterns

1. **Over-partitioning** small datasets
 2. **Collecting large datasets** to driver
 3. **Using Python UDFs** for simple operations
 4. **Ignoring data skew** in joins
 5. **Not caching** frequently accessed data
 6. **Using wrong join strategies** for data sizes
 7. **Inefficient serialization** (Java over Kryo)
 8. **Too many small files** in output
 9. **Not monitoring** resource utilization
 10. **Premature optimization** without profiling
-

Optimization Checklist

Data Layer

- ☐ Use columnar formats (Parquet/ORC)
- ☐ Implement proper partitioning strategy
- ☐ Enable predicate pushdown
- ☐ Optimize file sizes (128-256MB)
- ☐ Use bucketing for frequent joins

Execution Layer

- ☐ Enable Adaptive Query Execution
- ☐ Configure appropriate shuffle partitions
- ☐ Use vectorized UDFs when necessary
- ☐ Implement proper join strategies
- ☐ Enable dynamic partition pruning

Resource Layer

- ☐ Size executors appropriately
- ☐ Configure memory fractions
- ☐ Enable dynamic allocation
- ☐ Monitor resource utilization
- ☐ Use appropriate parallelism

Code Layer

- ☐ Use DataFrame API over RDD
 - ☐ Leverage built-in functions
 - ☐ Implement strategic caching
 - ☐ Handle data skew appropriately
 - ☐ Minimize shuffles
-

Performance Tuning Methodology

1. Measurement Phase

1. **Baseline Performance:** Measure current performance
2. **Identify Bottlenecks:** Use Spark UI and profiling
3. **Resource Utilization:** Monitor CPU, memory, I/O
4. **Data Patterns:** Analyze data distribution and access

2. Optimization Phase

1. **Prioritize Optimizations:** Focus on highest impact
2. **Implement Changes:** One optimization at a time
3. **Measure Impact:** Compare before/after performance
4. **Iterate:** Repeat for next optimization

3. Validation Phase

1. **Correctness:** Verify results remain accurate
 2. **Stability:** Test under various conditions
 3. **Scalability:** Validate with larger datasets
 4. **Monitoring:** Implement ongoing performance tracking
-

Summary

Spark optimization is an **iterative process** that requires understanding of:

- Data characteristics and access patterns
- Query execution plans and engine capabilities
- Resource allocation and cluster configuration
- Code-level optimizations and best practices

The key to successful optimization is **systematic measurement, targeted improvements, and continuous monitoring.**